



程序设计原理

结构体

任课教师：喻 梅
于瑞国
王建荣
李雪威
赵满坤



递归求最大公约数

- 欧几里得算法（辗转相除法）



扩展欧几里得

- 问题：假设你有一个5升的容器（容器A）和一个3升的容器（容器B）（以及充足的水源），如何精确地取出4升水来？
 - A装满水，倒入B将B装满，再将B的水全部倒出，继续将A的水倒入B，A空以后再将A接满，继续倒入B.....
 - 不断将A接满水倒入B，B满了就将B的水倒出，直至A中剩的水等于要求的水量
 - A接满2次水，B倒出2次水，最后A中剩的水为 $5*2-3*2=4$
- 容器A容量为a，容器B容量为b，需要取出c升水
 - 求 $ax+by=c$ 的整数解



扩展欧几里得

- 求不定方程 $ax+by=c$ 的整数解
- 裴蜀定理：对任何整数 a 、 b 和它们的最大公约数 d ，关于未知数 x 和 y 的线性不定方程（称为裴蜀等式）：若 a, b 是整数,且 $\gcd(a, b)=d$ ，那么对于任意的整数 $x, y, ax+by$ 都一定是 d 的倍数，特别地，**一定存在整数 x, y ，使 $ax+by=d$ 成立。**
- $ax+by=c$ 有整数解的充分必要条件是 $d=\gcd(a, b)$ 能整除 c
 - $ax+by=c$ 有整数解，设为 (x_1, y_1) ，则 $ax_1+by_1=c$ ，由于 d 为 a 和 b 的公约数，所以 d 也能整除 c
 - d 能整除 c ，设 $k=c/d$ ，**由于 $ax+by=d$ 有解**，设解为 (x_1, y_1) ，则 $ax_1+by_1=d$ ，两边同时乘以 k ，则 $akx_1+bky_1=c$ ，由于 kx_1 和 ky_1 都是整数，即 $ax+by=c$ 存在一组整数解 (kx_1, ky_1) 。



扩展欧几里得

- 证明：一定存在整数 x, y ，使 $ax+by=d$ 成立 ($d=\gcd(a,b)$)
- 欧几里得求最大公约数的过程为 $(a,b) \Rightarrow (b, a \% b) \Rightarrow \dots \Rightarrow (d, 0)$ 的过程
- 当 $a=d, b=0$ 时，即对于 $(d, 0)$ 这组数， $d*x+0*y=d$ 存在整数解 $x=1, y=\text{任意整数}$ ，使得 $ax+by=d$ 成立。如果能证明下面这个命题成立，那么就可以推导出 $ax+by=d$ 有整数解
- 证明：如果 $bx+(a \% b)y=d$ 有整数解，那么 $ax+by=d$ 有整数解
 - 设 (x_1, y_1) 为 $bx+(a \% b)y=d$ 的整数解， $a \% b$ 可表示为 $a-q*b$ ，其中 q 为 a/b 的下取整值，代入得 $bx_1+(a-qb)y_1=d$ ，转换得 $ay_1+b(x_1-ky_1)=d$ ，由于 $y_1, (x_1-ky_1)$ 都是整数，所以 (y_1, x_1-ky_1) 是 $ax+by=d$ 的一组整数解，即 $ax+by=d$ 有整数解



扩展欧几里得

- 代码实现

- $(y_1, x_1 - qy_1)$ 是 $ax + by = d$ 的一组整数解

- 递归过程中令 $x = y_1, y = x_1 - qy_1$, 其中 $q = a/b$ 的下取整值

```
int gcd(int a, int b) {  
    return b == 0 ? a : gcd(b, a%b);  
}
```

```
void ex_gcd(int a, int b, int &x, int &y) {  
    if(b == 0) {  
        x = 1; y = 0;  
    } else {  
        ex_gcd(b, a%b, x, y);  
        int x1 = x, y1 = y;  
        x = y1;  
        y = x1 - (a/b)*y1;  
    }  
}
```

```
int ex_gcd(int a, int b, int &x, int &y) {  
    if(b == 0) {  
        x = 1; y = 0;  
        return a;  
    } else {  
        int d = ex_gcd(b, a%b, x, y);  
        int x1 = x, y1 = y;  
        x = y1;  
        y = x1 - (a/b)*y1;  
        return d;  
    }  
}
```



扩展欧几里得

■ 扩展欧几里得

- 容器A容量为a, 容器B容量为b, 需要取出c升水

- 求 $ax+by=c$ 的整数解
- c需要是a和b最大公约数的倍数且 $c \leq \max(a,b)$
- 最少接水倒水次数? $|x|+|y|$ 最小 $\Rightarrow$ 接水次数最小
- 若 $a>b$, 求 $x>0$ 的最小的解; 否则, 求 $y>0$ 的最小的解
- $ax+by=c$ 的通解?
 - 若 x_0, y_0 为其一组特解
 - 则其通解为 $x = x_0 - (b/d)t, y = y_0 + (a/d)t$, t 为任意整数

定理2 设方程 $ax + by = c$ (其中 a, b 为非零整数) 有一组整数解 $x = x_0, y = y_0$, 则方程的所有整数解可以表示为 $x = x_0 - \frac{b}{\gcd(a,b)}t, y = y_0 + \frac{a}{\gcd(a,b)}t$. (1)

证明 已知 $ax_0 + by_0 = c$, 因此

$$a(x_0 - \frac{b}{\gcd(a,b)}t) + b(y_0 + \frac{a}{\gcd(a,b)}t) = ax_0 + by_0 = c.$$

从而对任意整数 t , (1) 式是方程的解.

反之, 设 x', y' 是方程的任一解, 则有 $ax' + by' = c = ax_0 + by_0$,

$$从而 a(x' - x_0) + b(y' - y_0) = 0,$$

$$即 \frac{a}{\gcd(a,b)}(x' - x_0) = -\frac{b}{\gcd(a,b)}(y' - y_0). (2)$$

所以 $\frac{a}{\gcd(a,b)}$ 整除 $\frac{b}{\gcd(a,b)}(y' - y_0)$, 又 $\frac{a}{\gcd(a,b)}$ 与 $\frac{b}{\gcd(a,b)}$ 互素,

所以 $\frac{a}{\gcd(a,b)}$ 整除 $(y' - y_0)$. 即存在整数 t , 使 $y' = y_0 + \frac{a}{\gcd(a,b)}t$.

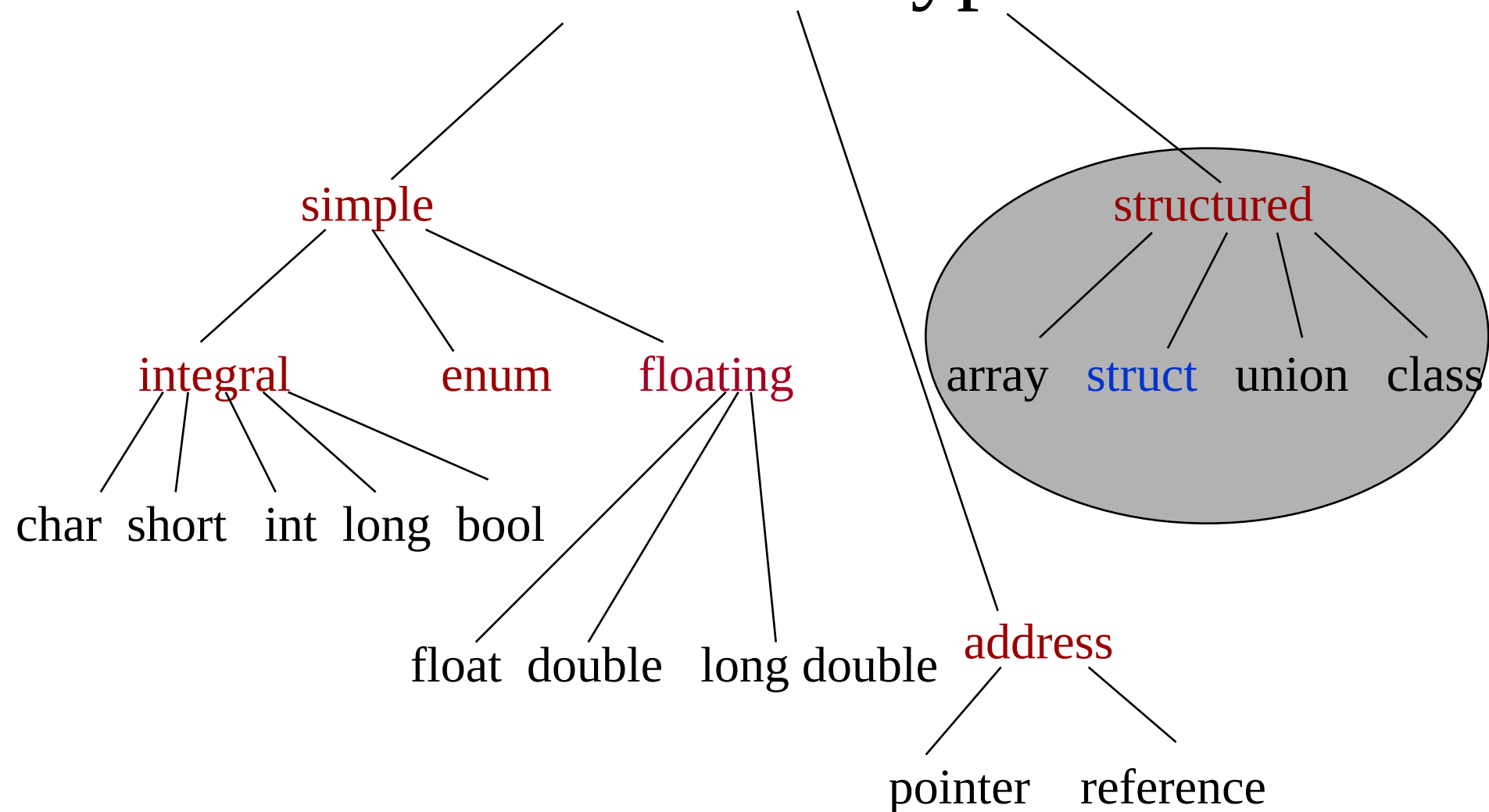
$$代入 (2) 得 x' = x_0 - \frac{b}{\gcd(a,b)}t.$$

故 (1) 式表示方程的所有整数解.



Structured Data Type

C++ Data Types





Structured Data Type

A **structured** data type is a type in which each value is a collection of component items

- The entire collection has a single name
- Each component can be accessed individually
- Used to bundle together related data of various types for convenient access under the same identifier

For example . . .



Structured Data Type

thisAnimal

5000

.id	2037581
.name	“giant panda”
.genus	“Ailuropoda”
.species	“melanoleuca”
.country	“China”
.age	18
.weight	234.6
.health	“Good”





Structured Data Type

anotherAnimal

6000

.id	5281003
.name	"llama"
.genus	"Lama"
.species	"peruana"
.country	"Peru"
.age	7
.weight	278.5
.health	"Excellent"



驼羊 Llama



Struct AnimalType

```
struct AnimalType // Declares a struct data type
{                // does not allocate memory
    long    id;
    string  name;
    string  genus;
    string  species;
    string  country;
    int     age;
    float   weight;
    string  health;
};
// Declare variables of AnimalType
AnimalType thisAnimal;
AnimalType anotherAnimal;
```

struct members



Struct Type Declaration

The struct declaration names a type and names the members of the struct

It **does not allocate memory for any variables of that type!**

You still need to declare your struct variables



More about struct type declarations

Scope of a struct

- If the struct type declaration precedes all functions, it will be visible throughout the rest of the file
- If it is placed within a function, only that function can use it

It is common to place struct type declarations in a (.h) header file and #include that file

It is possible for members of *different* struct types to have the same identifiers; also a non-struct variable may have the same identifier as a structure member



Struct Type Declaration

SYNTAX

```
struct TypeName           // Does not allocate memory
{
    MemberList
};
```

MemberList SYNTAX

```
DataType MemberName;
DataType MemberName;
.
.
.
```



Accessing struct Members

Dot (period) is the member selection operator

After the struct type declaration, the various members can be used in your program only when they are preceded by a struct variable name and a dot

EXAMPLES

thisAnimal.weight

anotherAnimal.country



Operations on struct Members

The type of the member determines the allowable operations

```
thisAnimal.age = 18;  
thisAnimal.id = 2037581;  
cin >> thisAnimal.weight;  
getline (cin, thisAnimal.species);  
thisAnimal.name = "giant panda";  
thisAnimal.genus[0] = toupper(thisAnimal.genus[0]);  
thisAnimal.age++;
```



Aggregate struct Operations

- **Operations valid on struct type variables are**
 - Assignment to another struct variable of the same type
 - Pass as an argument (by value or by reference)
 - Return as value of a function

- **I/O, arithmetic, and comparisons of entire struct variables are NOT ALLOWED!**



Aggregate struct Operations

```
anotherAnimal = thisAnimal;           // Assignment  
  
WriteOut(thisAnimal);                 // Value parameter  
  
ChangeWeightAndAge(thisAnimal);       // Reference parameter  
  
thisAnimal = GetAnimalData();         // Function return value
```



Aggregate struct Operations

```
void WriteOut( /* in */ AnimalType  thisAnimal)
// Prints out values of all members of thisAnimal
// Precondition: all members of thisAnimal are assigned
// Postcondition: all members have been written out
{
    cout << "ID # " << thisAnimal.id
          << thisAnimal.name << endl;

    cout << thisAnimal.genus << thisAnimal.species
          << endl;

    cout << thisAnimal.country << endl;

    cout << thisAnimal.age << " years " << endl;

    cout << thisAnimal.weight << " lbs. " << endl;

    cout << "General health : " << thisAnimal.health << endl;
}
```



Passing a struct Type by Reference

```
void ChangeAge(/* inout */ AnimalType& thisAnimal)
// Adds 1 to age
// Precondition: thisAnimal.age is assigned
// Postcondition: thisAnimal.age ==
//   thisAnimal.age@entry + 1
{
    thisAnimal.age++;
}
```



Return a struct Type

AnimalType GetAnimalData ()

```
// Obtains all information about an animal from keyboard
// Postcondition:
//   Return value == AnimalType members entered at kbd
{
    AnimalType  thisAnimal;
    char response;
    do
    {
        // Have user enter members until they are correct
        .
        .
        .
    } while (response != 'Y');
    return  thisAnimal;
}
```



扩展欧几里得

- 定义一个结构体，包括需要返回的三个元素 x, y, d ，将结构体类型作为函数的返回值类型

```
struct ANS {  
    int x,y,d;  
};  
ANS ex_gcd(int a,int b) {  
    if(b == 0) {  
        ANS ans = {1,0,a};  
        return ans;  
    } else {  
        ANS ans = ex_gcd(b,a%b);  
        int tmpx = ans.x;  
        ans.x = ans.y;  
        ans.y = tt - (a/b)*ans.y;  
        return ans;  
    }  
}
```



Hierarchical Structures

- **The type of a struct member can be another struct type**
- **This is called nested or hierarchical structures**
- **Hierarchical structures are very useful when there is much detailed information in each record**

For example . . .



struct MachineRec

Information about each machine in a shop contains:

an idNumber,

a written description,

the purchase date,

the cost,

and a history (including failure rate, number of days down, and date of last service)



struct MachineRec

```
struct DateType
{
    int    month;           // Assume 1 . . 12
    int    day;             // Assume 1 . . 31
    int    year;            // Assume 1900 . . 2050
};
struct StatisticsType
{
    float  failRate;
    DateType  lastServiced; // DateType is a struct type
    int  downDays;
};
struct MachineRec
{
    int  idNumber;
    string  description;
    StatisticsType  history; // StatisticsType is a struct
    DateType  purchaseDate;
    float  cost;
};
MachineRec  machine;
```



struct type variable machine

7000

5719	“DRILLING...”	<table><tr><td>.02</td><td><table><tr><td>1</td><td>25</td><td>2025</td></tr><tr><td>.month</td><td>.day</td><td>.year</td></tr></table></td><td>4</td></tr><tr><td>.failrate</td><td>.lastServiced</td><td>.downdays</td><td></td></tr></table>	.02	<table><tr><td>1</td><td>25</td><td>2025</td></tr><tr><td>.month</td><td>.day</td><td>.year</td></tr></table>	1	25	2025	.month	.day	.year	4	.failrate	.lastServiced	.downdays		<table><tr><td>3</td><td>21</td><td>1995</td></tr><tr><td>.month</td><td>.day</td><td>.year</td></tr></table>	3	21	1995	.month	.day	.year	8000.0
.02	<table><tr><td>1</td><td>25</td><td>2025</td></tr><tr><td>.month</td><td>.day</td><td>.year</td></tr></table>	1	25	2025	.month	.day	.year	4															
1	25	2025																					
.month	.day	.year																					
.failrate	.lastServiced	.downdays																					
3	21	1995																					
.month	.day	.year																					
.purchaseDate	.cost																						

.idNumber .description

. history

.purchaseDate

.cost

machine.history.lastServiced.year has value 2025



Exercise

TJU某个学院的大一新生在本学期修了五门课程，这五门课程分别有着相应的学分，现在辅导员想要在本学期结束时计算自己所带的 N 个学生在本学期的加权成绩并对加权成绩进行排序（如遇加权成绩相同的情况，那么按照从学分高的课程到学分低的课程，依次比较单科成绩，单科成绩高的同学靠前）。

例如：假设 5 门课的学分依次为 6, 5, 3, 2, 1; mfc 同学的成绩分别是 90, 80, 90, 92, 90, mdz 同学的成绩分别是 90, 80, 90, 90, 94。显然两个同学的加权成绩是一样的，同时两个同学学分为 6, 5, 3 的三门课程成绩都相同，但 mfc 同学在学分为 2 的课程中，成绩优于 mdz 同学，故 mfc 同学的排名略优于 mdz 同学。

再了解了具体的规则后，身为计算机专业的学生，你能够帮助该辅导员处理该问题吗？

加权成绩 $S = \sum_{i=1}^5 s_i * \frac{p_i}{\sum_{j=1}^5 p_j}$ (其中 s 代表对应科目成绩， p 代表该科目对应学分， $\sum_{j=1}^5 p_j$ 代表 5 门课程的总学分)。



Exercise

样例输入 1



```
6 5 3 2 1
10
student1 98 97 62 91 74
student2 78 63 76 63 98
student3 80 97 66 62 95
student4 74 61 81 73 92
student5 85 76 87 94 67
student6 87 65 84 79 69
student7 61 88 89 71 95
student8 83 80 79 63 98
student9 70 61 94 80 85
student10 88 76 67 94 79
```

样例输出 1



```
student1 89.12
student5 82.71
student3 81.29
student10 80.94
student8 79.94
student6 78.00
student7 77.06
student9 73.65
student2 72.65
student4 72.35
```